

Rapport de Projet : Détection de communautés dans DBLP

Algorithmique 2 (INFO-F-203)

Machado Ozcifei Mikail Zeki (Matricule : 592868)

Adiya Chinzorig (Matricule : 587607)

Année Académique 2025-2026

1 Introduction

Dans le cadre de ce projet, nous analysons la base de données bibliographiques DBLP afin d'y identifier des communautés de chercheur·euse·s. Chaque publication lue nous fournit une liste d'auteur·rice·s que nous modélisons sous forme de graphes. L'objectif principal est de concevoir des algorithmes capables de traiter ces données en "online" pour calculer et maintenir ces communautés.

Pour ce faire, nous mettons en œuvre les algorithmes étudiés au cours, en les adaptant aux spécificités de nos deux tâches : la recherche de composantes connexes sur un graphe non orienté, et l'identification de composantes fortement connexes sur un graphe orienté.

2 Méthodologie

2.1 Algorithmes et structures de données

2.1.1 Tache 1

Cette tâche demande de calculer et de maintenir à jour les composantes connexes d'un graphe non orienté, ainsi que la taille de ces communautés.

- **Structure principale** : Étant donné que le tracé exact des relations entre auteurs n'a pas d'importance ici (seules les composantes comptent), une structure *Union-Find* est parfaite pour ce problème. Une `HashMap` est utilisée pour faire le lien entre le nom d'un·e auteur·rice (chaîne de caractères) et son identifiant (entier) dans la structure.
- **Optimisations** : Pour que le programme reste rapide malgré le très grand nombre de fusions à réaliser, notre *Union-Find* utilise la compression des chemins et l'union pondérée.
- **Maintien des données "online"** : Un tableau supplémentaire permet de garder en mémoire la taille des différentes communautés au fur et à mesure de la lecture. Tout comme pour le fonctionnement de l'*Union-Find*, seule la valeur associée à la racine d'une communauté a une signification.
- **Calculs ponctuels** : L'histogramme et la taille des 10 plus grandes communautés ne sont pas mis à jour en continu. Ils sont simplement calculés à l'aide d'un parcours linéaire des tableaux au moment où l'on doit afficher les résultats.

2.1.2 Tache 2

- **Traitement "online"** : La lecture du fichier XML s'effectue de manière séquentielle via la classe `DblpPublicationGenerator`. Les relations asymétriques et leurs occurrences sont comptabilisées à la volée dans une `HashMap<String, HashMap<String, Integer>` afin de minimiser l'impact mémoire avant l'étape de filtrage.

- **Détection des SCC** : Une fois le graphe orienté construit et filtré (seuil ≥ 6), nous y appliquons l'algorithme de Kosaraju. Celui-ci nécessite deux parcours en profondeur (DFS) successifs : le premier détermine l'ordre de fin de traitement, le second, opéré sur le graphe inversé, extrait les composantes fortement connexes.
- **Calcul du diamètre** : Pour calculer le diamètre d'une communauté donnée, nous lançons un parcours en largeur (BFS) depuis chaque nœud la composant. Cela nous permet d'isoler la plus longue distance de plus court chemin entre deux auteur·rice·s en restant strictement au sein de la SCC.

2.2 Complexité

2.2.1 Tache 1

Dans l'analyse suivante, nous considérons le pire des cas, avec V le nombre d'auteurs uniques ajoutés à la structure.

- **Temps** :
 - **Union** : $O(\log V)$. La structure *Union-Find* optimisée offre une complexité quasi constante, mais la recherche préalable des identifiants dans la `HashMap` se fait en temps logarithmique dans le pire des cas.
 - **Ajout d'un auteur** : $O(V)$. Bien que l'ajout se fasse en temps quasi-constant, le pire des cas correspond au moment où les tableaux internes de l'*Union-Find* sont pleins et doivent être entièrement copiés pour être agrandis.
 - **Extraction des résultats** : $O(V)$. La création de l'histogramme et du top 10 demande un parcours linéaire du tableau des tailles. La file de priorité (*Priority Queue*) utilisée pour le top 10 ayant une taille maximale constante de 10, ses opérations d'insertion coûtent $O(\log 10) = O(1)$.
- **Mémoire** : $O(V)$. La `HashMap` servant de dictionnaire et les tableaux internes de l'*Union-Find* (parents, tailles et hauteurs) prennent tous un espace directement proportionnel au nombre d'auteurs enregistrés.

2.2.2 Tache 2

Dans l'analyse suivante, nous considérons le pire des cas, avec V le nombre d'auteurs et E le nombre de relations orientées conservées **après le filtrage** (seuil ≥ 6 publications).

- **Temps** :
 - **Détection des SCC** : $O(V + E)$. L'algorithme de Kosaraju nécessite deux parcours en profondeur (DFS) successifs sur l'ensemble du graphe.
 - **Calcul du diamètre** : $O(V_{scc} \cdot (V_{scc} + E_{scc}))$ pour chaque communauté. Cela correspond au lancement complet d'un parcours en largeur (BFS) depuis chaque sommet V_{scc} appartenant à cette composante.
- **Mémoire** : $O(V + E)$. Cet espace est nécessaire pour stocker les listes d'adjacence du graphe (ainsi que sa version inversée) et les structures annexes permettant de marquer les nœuds visités lors des parcours.

3 Résultats

3.1 Tache 1

3.1.1 Histogramme

Voici la distribution des tailles de la Tache 1

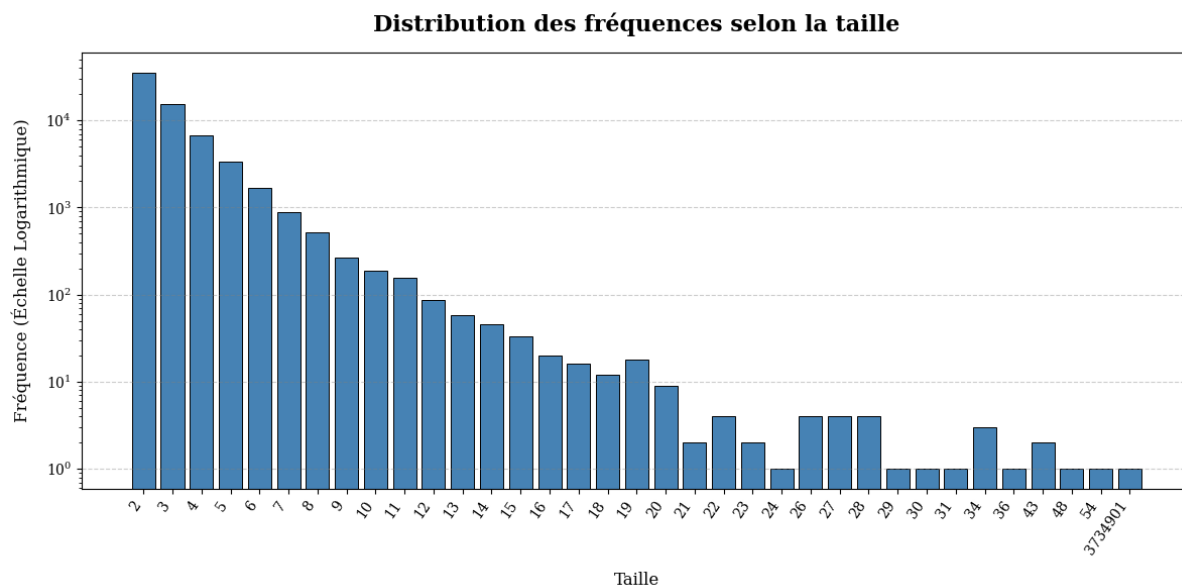


FIGURE 1 – Distribution de la taille des communautés de la tâche 1

3.2 Tache 2

3.2.1 Top 10 des communautés

Le Tableau 1 présente les caractéristiques des 10 plus grandes communautés identifiées de la tâche 2.

Rang	Taille (Membres)	Diamètre
1	[133]	[30]
2	[73]	[35]
3	[38]	[13]
4	[34]	[7]
5	[33]	[6]
6	[30]	[17]
7	[27]	[10]
8	[26]	[12]
9	[23]	[6]
10	[21]	[7]

TABLE 1 – Analyse structurale des 10 plus grandes SCC de la tâche 2.

3.2.2 Distribution des tailles

L’histogramme des tailles de communautés suit une progression logarithmique, illustrant la prédominance de très petites structures de collaboration par rapport à quelques grands pôles

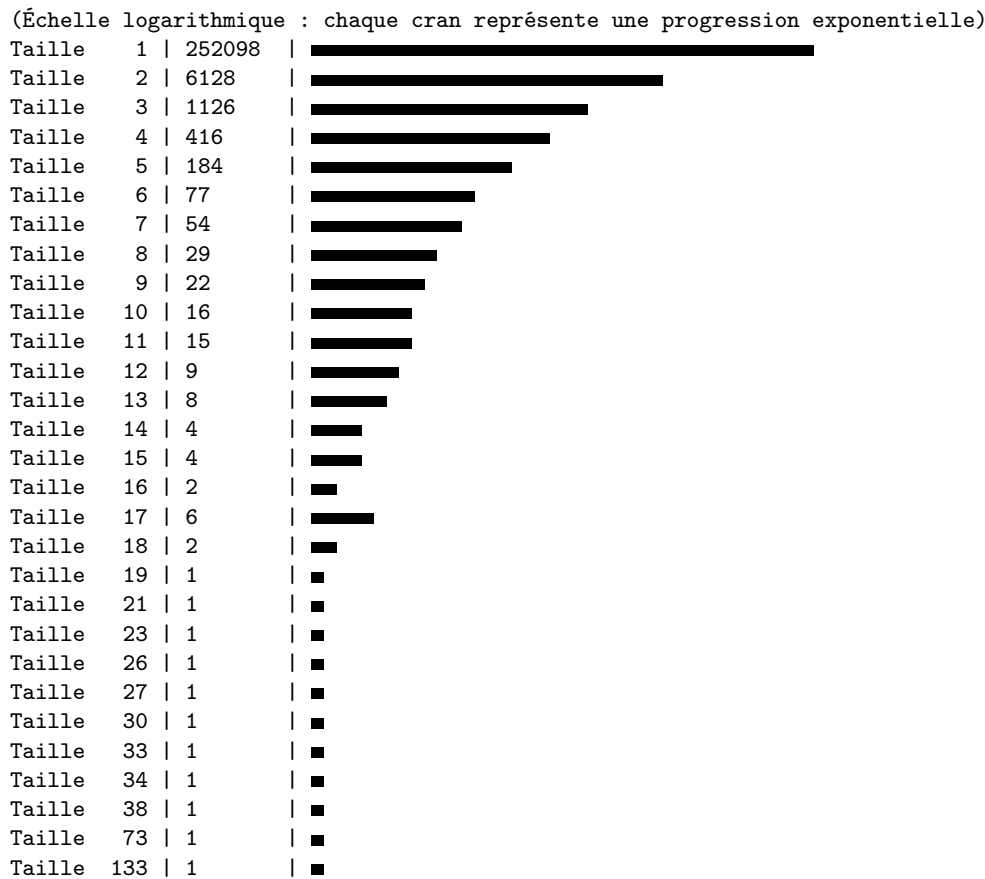


FIGURE 2 – Distribution des tailles de communautés (sortie brute du programme tache 2).

de recherche. la Figure 1 présente l’histogramme des tailles de communautés. Le programme utilise une HashMap imbriquée (`Map<Integer, Map<Integer, Integer>`) pour comptabiliser les collaborations entre auteurs de manière efficace. Cette structure permet de filtrer rapidement les relations tout en optimisant l’usage de la mémoire.

4 Note sur l’utilisation d’outils d’assistance

L’implémentation technique de cette tâche a été optimisée à l’aide d’une assistance par intelligence artificielle sur deux points précis :

- **Syntaxe et API Java** : L’usage des expressions **lambda** et des méthodes de l’interface **Map** a été optimisé pour garantir une gestion concise et performante des compteurs de co-autorat.
- **Représentation visuelle** : La logique de calcul pour l’affichage de l’histogramme en sortie console utilise une **échelle logarithmique**. Cette approche permet de visualiser efficacement la distribution des fréquences malgré la prédominance statistique des auteurs uniques.

5 Discussion

- **Limites** : L’implémentation actuelle utilise des objets **String** en mémoire vive pour stocker les noms d’auteurs. Cela limite la montée en charge sur des bases de données plus massives.

- **Optimisations (OpenAlex)** : Pour traiter 120 millions d’auteurs, il serait nécessaire d’indexer les auteurs par des entiers (`int`) et d’utiliser du stockage externe (disque) pour la phase de comptage des paires. Le calcul du diamètre devrait également être approximé pour les très grandes composantes.

6 Conclusion

Ce projet a permis d’extraire des communautés scientifiques cohérentes à partir de DBLP en utilisant des algorithmes de graphes fondamentaux. L’approche par SCC se révèle efficace pour isoler des groupes de chercheurs aux collaborations mutuelles fortes et fréquentes.